

《基于 openEuler 的操作系统分析与 设计实习》 指导书

华南农业大学数学与信息学院

2020.12

目 录

第一部分 《操作系统分析与设计实习》要求	2
一、《操作系统分析与设计实习》教学概述	2
1、教学基本情况	2
2、教学指导思想和教学目的	2
3、项目表	2
二、《操作系统分析与设计实习》教学规范	2
1、课程意义	2
2、实验步骤	3
3、课程设计报告(文档)规范	4
4、课程考核	6
第二部分 《操作系统分析与设计实习》内容	7
系统操作类题目（必做）	7
题目一 OpenEuler操作系统安装	7
题目二 操作系统用户管理	13
题目三 基于OpenEuler操作系统的应用开发	16
题目四 Docker技术的应用	17
编程设计类题目（从下列几个题目中选）	21
题目一 进程同步与互斥	21
题目二 页面置换算法的模拟实现及命中率对比	21
题目三 单处理器系统的进程调度	25
题目四 磁盘调度算法的模拟实现及对比	25
题目五 模拟磁盘文件系统实现	30
题目六 模拟操作系统实现	37

第一部分 《操作系统分析与设计实习》要求

一、《操作系统分析与设计实习》教学概述

1、教学基本情况

课程总学时数：2 周； 课程总学分：2 学分

适用专业：信息学院计算机科学与技术、软件工程、网络工程专业，软件学院软件工程专业

考核方式及方法：实际操作+程序运行+课程设计报告。

成绩评定：在参考“难度系数”的基础上

>=90——功能完善，编程风格好，人机接口界面好，报告结构清晰，总结和分析详尽；

80~90——功能完善，编程风格良好，人机接口界面良好，报告结构完整；

70~80——功能基本完善，编程风格良好，报告结构完整；

60~70——能完成所选题目，并提交报告；

<60——未按时完成，或者抄袭（含雷同者）。

2、教学指导思想和教学目的

1) 指导思想：通过由浅入深、循序渐进、精讲多练，培养学生对计算机操作系统的熟练使用，使学生全面了解操作系统的特点，熟练掌握操作系统的基本设计方法和系统工作原理。

2) 教学目的：使学生通过实践来验证课堂教学的理论，并学会设计一些简单的综合应用程序或小型的模拟操作系统。

3、项目表

一共分为两部分实验，第一部分属于 openEuler 系统操作题，提供四个题目，必做实验；第二部分为编程设计类题目，共 6 个题目，难度逐级递增。第一个题目难度最小，第 6 个题目难度最大。前四个题目需要单独完成，后两个题目可组队完成，其中第五个题目建议 2 到 3 人一队，第 6 个题目建议 3 到 5 人一队。每位同学根据自己的能力选择一个题目。

不要求对真实操作系统的各种数据结构进行操作，而是对你自己所设置的一些数据结构（如数组、链表或队列）进行操作，来模拟实现操作系统中的算法或调度行为。你可以采用任何你熟悉的语言编写。

二、《操作系统分析与设计实习》教学规范

1、课程意义

本课程是对学生的一种全面综合训练，是与课堂听讲、自学和练习相辅相成的必不可少的一个教学环节。通常，实验所涉及到的问题比平时的习题复杂得多，也更接近实际。本课程着眼于原理与应用的结合点，使学生学会如何把书上学到的知识用于解决实际问题，培养软件工作所需要的动手能力；另一方面，能使书上的知识变“活”，起到深化理解和灵活掌握教学内容的目的。平时的练习较偏重于如何编

写功能单一的“小”算法，而本课程涉及到的实验题目是软件设计的综合训练，包括问题分析、总体结构设计、用户界面设计、程序设计基本技能和技巧，多人合作，以至一整套软件工作规范的训练和科学作风的培养。此外，还有很重要的一点是：机器是比任何教师都严厉的检查者。

本课程的革新之处在于采用华为鲲鹏服务器平台，基于自主研发的 openEuler 操作系统，完成一系列独立实验，经过此过程使学生能够清楚地了解国产操作系统的操作特点以及优势，同时深刻理解操作系统的基本原理。

2、实验步骤

常用的软件开发方法，是将软件开发过程划分为分析、设计、实现和维护四个阶段。虽然本课程所涉及到的实验题目远不如实际问题中的复杂程度高，但为了培养一个软件工作者所应具备的科学工作的方法和作风，也应遵循以下五个步骤来完成实验题目：

1) 问题分析和任务定义

在进行设计之前，首先应该充分地分析和理解问题，明确问题要求做什么，限制条件是什么。本步骤强调的是做什么，而不是怎么做。对问题的描述应避开算法和所涉及的数据类型，而是对所需完成的任务做出明确的回答。例如：输入数据的类型、值的范围以及输入的形式；输出数据的类型、值的范围及输出的形式；若是会话式的输入，则结束标志是什么，是否接受非法的输入，对非法输入的回答方式是什么等。还应该为调试程序准备好测试数据，包括合法的输入数据和非法形式的输入数据。

2) 逻辑设计和详细设计

在设计这一步骤中需分逻辑设计和详细设计两步实现。逻辑设计指的是，对问题描述中涉及的操作对象定义相应的数据类型，并按照以数据结构为中心的原则划分模块，定义主程序模块和各抽象数据类型；详细设计则为定义相应的存储结构并写出各函数的伪码算法。在这个过程中，要综合考虑系统功能，使得系统结构清晰、合理、简单和易于调试，抽象数据类型的实现尽可能做到数据封装，基本操作的规格说明尽可能明确具体。作为逻辑设计的结果，应写出每个抽象数据类型的定义(包括数据结构的描述和每个基本操作的功能说明)，各个主要模块的算法，并画出模块之间的调用关系图。详细设计的结果是对数据结构和基本操作作出进一步的求精，写出数据存储结构的类型定义，写出函数形式的算法框架。在求精的过程中，应尽量避免陷入语言细节，不必过早表述辅助数据结构和局部变量。

3) 编码实现和静态检查

编码是把详细设计的结果进一步求精为程序设计语言程序。如果基于详细设计的伪码算法就能直接在键盘上输入程序的话，则可以不必用笔在纸上写出编码，而将这一步的工作放在上机准备之后进行，即在上机调试之前直接用键盘输入。

然而，不管你是否写出编码的程序，在上机之前，认真的静态检查是必不可少的。静态检查主要有两种方法，一是用一组测试数据手工执行程序(通常应先分模块检查)；二是通过对程序深入全面地理解程序逻辑，在这个过程中再加入一些注解和断言。如果程序中逻辑概念清楚，后者将比前者有效。

4) 上机准备和上机调试

上机环境：所有实验需要远程登录华为鲲鹏服务器平台系统，基于 openEuler 操作系统完成。通过 Docker 技术，每组同学独立实现设计题目。

上机准备：需要熟悉 openEuler 操作系统的安装使用相关基础知识；需要熟练运用 Linux 操作命令；需要熟悉 Docker 操作方法以及部分脱机实验程序的移植技巧。

注意事项：

- (1) 注意同一高级语言文本之间的差别。
- (2) 熟悉机器的操作系统和语言集成环境的用户手册，尤其是最常用的命令操作，以便顺利进行上机的基本活动。
- (3) 掌握调试工具，考虑调试方案，设计测试数据并手工得出正确结果。应该能够熟练运用高级语言的程序调试器 DBBUG 调试程序
- (4) 上机调试程序时要带一本高级语言教材或手册。调试最好分模块进行，自底向上，即先调试底层函数。在调试过程中可以不断借助 DEBUG 的各种功能，提高调试效率。调试中遇到的各种异常现象往往是预料不到的，此时应动手确定疑点，通过修改程序来证实它或绕过它。调试正确后，认真整理源程序及其注释，形成格式和风格良好的源程序清单和结果。

5) 总结和整理实验报告

3、课程设计报告(文档)规范

报告的开头应首先包括如下成绩单表格，并填写班级、学号、姓名、题目等信息。在成绩单之后另起一新页，开始你的报告主体。

华南农业大学数学与信息（软件）学院

《操作系统分析与设计实习》成绩单

开设时间：2020 学年第二学期

小组成员、组内分工及各成员成绩							
学号		姓名		分工		成绩	
学号		姓名		分工		成绩	
学号		姓名		分工		成绩	
实验 题目							
自我 评价							
教师 评语	评价指标： ● 题目内容和要求完成情况 优 ☐ 良 ☐ 中 ☐ 差 ☐ ● 对算法原理的理解程度 优 ☐ 良 ☐ 中 ☐ 差 ☐ ● 程序设计水平 优 ☐ 良 ☐ 中 ☐ 差 ☐ ● 程序运行效果及正确性 优 ☐ 良 ☐ 中 ☐ 差 ☐ ● 课程设计报告结构清晰 优 ☐ 良 ☐ 中 ☐ 差 ☐ ● 报告中总结和分析详尽 优 ☐ 良 ☐ 中 ☐ 差 ☐						
				教师签名			

然后，在报告主体中包括以下六个内容：

一、需求分析

明确陈述说明程序设计的任务，强调的是程序要做什么，主要包括：

- (1) 输入的形式和输入值的范围；
- (2) 输出的形式；
- (3) 程序所能达到的功能；
- (4) 测试数据：包括正确的输入及其输出结果和含有错误的输入及其输出结果。

二、概要设计

说明本程序中用到的所有抽象数据类型的定义、主程序的流程以及各程序模块之间的层次(调用)关系。

三、详细设计

实现概要设计中定义的所有数据类型，对每个操作只需要写出伪码算法；对主程序和其他模块也都需要写出伪码算法(伪码算法达到的详细程度应能够按照伪码算法在计算机键盘上直接输入高级程序设计语言程序)；画出函数的调用关系图。

四、调试分析

内容包括：

- (1) 调试过程中遇到的问题是如何解决的以及对设计与实现的讨论和分析；
- (2) 算法的时间复杂性(包括基本操作和其他算法的时间复杂性的分析)和改进设想；
- (3) 设计过程的经验和体会；
- (4) 实现过程中出现的主要问题及解决方法。

五、用户使用说明

说明如何使用你编写的程序，详细列出每一步的操作步骤。

六、测试与运行结果

列出你的测试结果和运行情况(即运行时的关键画面)，包括输入和输出。这里的测试数据应该完整和严格，最好多于需求分析中所列。

值得注意的是，报告的各种文档资料，要在程序开发的过程中逐渐充实形成，而不是最后补写。必要时可在报告中附部分关键源代码，但不需要附全部源代码。

4、课程考核

1) 考核点：编程(50分)、测试分析(20分)、报告(30分)；

2) **上交内容(电子版)：**

①**源代码；**

②**可执行程序；**

③**课程设计报告。**

上交时间与地点由任课教师指定。**请各班学习委员注意：**上交电子版文件时，每位或每组同学的内容各自放在单独的一个文件夹中(此文件夹不要压缩)，文件夹的名字格式形如：**201231000101 陈陈**，即只包含学号姓名，且学号和姓名之间没有任何空格及其它符号。最后整个班压缩为一个文件发送给任课教师。

第二部分 《操作系统分析与设计实习》内容

系统操作类题目（必做）

题目一 OpenEuler 操作系统安装

实验内容

1、设置安装语言，默认为英语，用户可根据实际情况进行调整，如图 1 所示，选择“中文”。

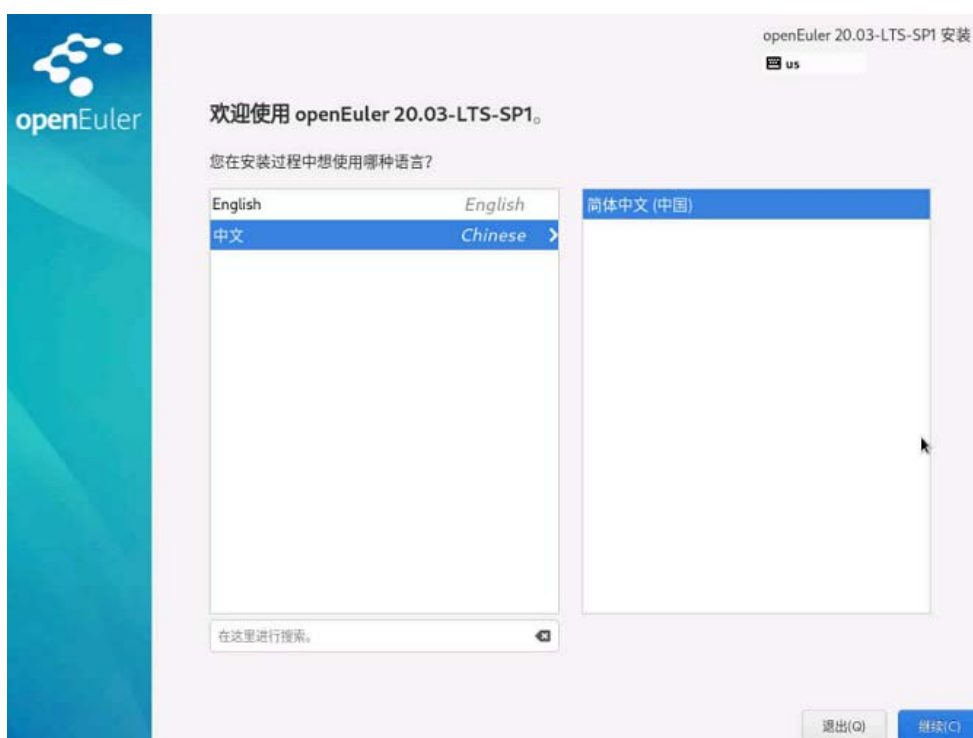


图 1 选择语言

2、在安装概览界面，根据实际情况设置各配置项。

- 1) 配置项有告警符号的，表示用户必须完成该选项配置后，告警符号消失，才能进行下一步操作。
- 2) 配置项无告警符号的，表示该配置项已有默认配置。
- 3) 所有配置项均无告警符号时用户才能单击“开始安装”进行系统安装。



图 2 安装概览

3、选择“软件选择”，设置“软件选择”配置项。

用户需要根据实际的业务需求，在左侧选择一个“最小安装”，在右侧选择安装环境的附加选项，如图 3 所示。

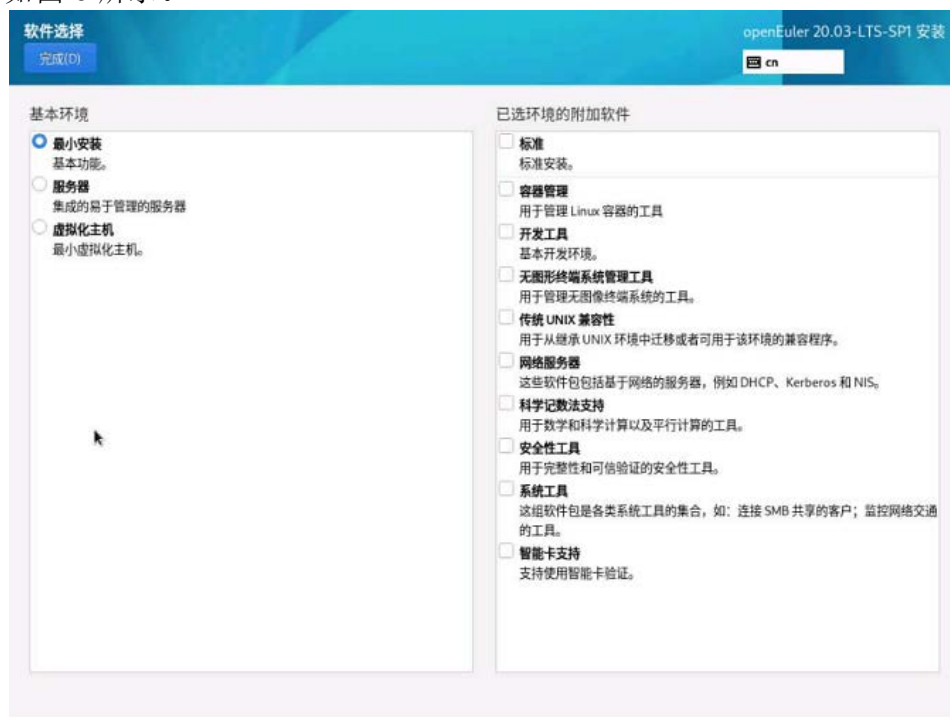


图 3 软件选择

提示：

- 1) 在最小安装的环境下，并非安装源中所有的包都会安装。如果用户需要使用的包未安装，可将安装源挂载到本地制作 **repo** 源，通过 **DNF** 工具单独安装。
 - 2) 选择“虚拟化主机”时会默认安装虚拟化组件 **qemu**、**libvirt**、**edk2**，且可在附件选项处选择是否安装 **ovs** 等组件。
 - 3) 设置完成后，请单击左上角“完成”返回“安装概览”页面。
- 4、选择“安装目的地”，设置“安装目的地”配置项。

在安装位置页面中，选择计算机中的本地可用存储设备。

须知：

- 1) 在选择您需要安装的设备时，建议不要选择 NVMe SSD 存储介质作为操作系统的安装磁盘。
- 2) 您还需要进行存储配置以便对系统分区。您可以手动配置分区，也可以选择让安装程序自动分区。如果是在未使用过的存储设备中执行全新安装，或者不需要保留该存储设备中任何数据，建议选择“自动”进行自动分区。如图 4 所示。



图 4 安装目标位置

说明：

- 1) 在进行分区时，出于系统性能和安全的考虑，建议您划分如下单独分区：**/boot**、**/var**、**/var/log**、**/var/log/audit**、**/home**、**/tmp**。
 - 2) 系统如果配置了 **swap** 分区，当系统的物理内存不够用时，会使用 **swap** 分区。虽然 **swap** 分区可以增大物理内存大小的限制，但是如果由于内存不足使用到 **swap** 分区，会增加系统的响应时间，性能变差。因此在物理内存充足或者性能敏感的系统，不建议配置 **swap** 分区。
 - 3) 如果需要拆分逻辑卷组则需要选择“自定义”进行手动分区，并在“手动分区”界面单击“卷组”区域中的“修改”按钮重新配置卷组。
- 设置完成后，请单击左上角“完成”返回“安装概览”页面。

5、选择“根密码”，设置“根密码”配置项。

在“ROOT 密码”页面中，如图 5 所示，根据密码复杂度输入密码并再次输入密码进行确认。

说明：

- 1) root 帐户是用来执行关键系统管理任务，不建议您在日常工作及系统访问时使用 root 帐户。
- 2) 在“ROOT 密码”界面若选择“锁定 root 帐户”则 root 帐户将禁用。



图 5 root 密码

用户设置的 root 用户密码或新创建用户的密码均需要满足密码复杂度要求，否则会导致密码设置或用户创建失败。设置密码的复杂度的要求如下：

- 1) 口令长度至少 8 个字符。
- 2) 口令至少包含大写字母、小写字母、数字和特殊字符中的任意 3 种。
- 3) 口令不能和账号一样。
- 4) 口令不能使用字典词汇。

说明：可以通过 `cracklib-unpacker/usr/share/cracklib/pw_dict > dictionary.txt` 命令导出字典库文件 `dictionary.txt`，用户可以查询密码是否在该字典中。

设置完成后，单击左上角的“完成”返回“安装概览”页面。

6、选择“创建用户”，设置“创建用户”配置项。

在创建用户的界面如图 6 所示。输入用户名，并设置密码，其中密码复杂度要求与 root 密码复杂度要求一致。另外您还可以通过“高级”选项设置用户主目录、用户组等，如图 7 所示。



图 6 创建用户



图 7 高级用户配置

7、完成设置后，单击左上角的“完成”返回“安装概览”页面。
 设置其他配置项，其他配置项可以使用默认配置。
 单击“开始安装”进行系统安装，如图 8 所示。



图 8 开始安装

8、安装完成后重启系统。

openEuler 完成安装后，单击“重启”按钮，系统将重新启动。

题目二 操作系统用户管理

实验内容

在 Linux 中，每个普通用户都有一个账户，包括用户名、密码和主目录等信息。除此之外，还有一些系统本身创建的特殊用户，它们具有特殊的意义，其中最重要的是管理员账户，默认用户名是 root。同时 Linux 也提供了用户组，使每一个用户至少属于一个组，从而便于权限管理。

用户和用户组管理是系统安全管理的重要组成部分，下面介绍 openEuler 提供的用户管理和组管理命令，以及为普通用户分配特权的方法。

1、增加用户

在 root 权限下，通过 useradd 命令可以为系统添加新用户信息，其中 options 为相关参数，username 为用户名称：

```
useradd [options] username
```

2、创建用户实例

例如新建一个用户名为 userexample 的用户，在 root 权限下执行如下命令：

```
# useradd userexample
```

没有任何提示，表明用户建立成功。这时并没有设置用户的口令，请使用 passwd 命令修改用户的密码，没有设置密码的新账号不能登录系统。

使用 id 命令查看新建的用户信息，命令如下：

```
# id userexample
```

修改用户 userexample 的密码，命令如下：

```
# passwd userexample
```

修改用户密码时需要满足密码复杂度要求，密码的复杂度的要求如下：

- 1) 口令长度至少 8 个字符。
- 2) 口令至少包含大写字母、小写字母、数字和特殊字符中的任意 3 种。
- 3) 口令不能和账号一样。
- 4) 口令不能使用字典词汇。

3、修改用户账号

(1) 修改密码

普通用户可以用 `passwd` 修改自己的密码，只有管理员才能用 `passwd username` 为其他用户修改密码。

（2）修改主目录

修改主目录，可以在 `root` 权限下执行如下命令，其中 `new_home_directory` 为已创建的目标主目录的路径，`username` 为要修改用户的用户名，请根据实际情况修改：

```
usermod -d new_home_directory username
```

如果想将现有主目录的内容转移到新的目录，应该使用 `-m` 选项，命令如下：

```
usermod -d new_home_directory -m username
```

（3）管理员账户授权

使用 `sudo` 命令可以允许普通用户执行管理员账户才能执行的命令。

`sudo` 命令允许已经在 `/etc/sudoers` 文件中指定的用户运行管理员账户命令。例如，一个已经获得许可的普通用户可以运行如下命令：

```
sudo /usr/sbin/useradd newuser1
```

实际上，`sudo` 的配置完全可以指定某个已经列入 `/etc/sudoers` 文件的普通用户可以做什么，不可以做什么。

`/etc/sudoers` 的配置行如下所示：

- 1) 空行或注释行（以 `#` 字符打头）：无具体功能的行。
- 2) 可选的主机别名行：用来创建主机列表的简称。必须以 `Host_Alias` 关键词开头，列表中的主机必须用逗号隔开，如：

```
Host_Alias linux=ted1,ted2
```

其中 `ted1` 和 `ted2` 是两个主机名，可使用 `linux`（别名）称呼它们。

- 3) 可选的用户别名行：用来创建用户列表的简称。用户别名行必须以 `User_Alias` 关键词开头，列表中的用户名必须以逗号隔开。其格式同主机别名行。
- 4) 可选的命令别名行：用来创建命令列表的简称。必须以 `Cmnd_Alias` 开头，列表中的命令必须用逗号隔开。
- 5) 可选的运行方式别名行：用来创建用户列表的简称。不同的是，使用这样的别名可以告诉 `sudo` 程序以列表中某一用户的身份来运行程序。
- 6) 必要的用户访问说明行。

用户访问的说明语法如下：

```
user host = [ run as user ] command list
```

在 `user` 处指定一个真正的用户名或定义过的别名，`host` 也可以是一个真正的主机名或者定义过的主机别名。默认情况下，`sudo` 执行的所有命令都是以 `root` 身份执行。如果您想使用其他身份可以指定。`command list` 可以是以逗号分隔的命令列表，也可以是一个已经定义过的别名，如：

```
ted1 ted2=/sbin/shutdown
```

上面一句说明 `ted1` 可以在 `ted2` 主机上运行关机命令。

```
newuser1 ted1=(root) /usr/sbin/useradd,/usr/sbin/userdel
```

上面一句说明 ted1 主机上的 newuser1 具有以 root 用户权限执行 useradd,userdel 命令的功能。

说明：

- 1) 可以在一行定义多个别名，中间用冒号 (:) 隔开。
- 2) 可在命令或命令别名之前加上感叹号 (!)，使该命令或命令别名无效。
- 3) 有两个关键词：ALL 和 NOPASSWD。ALL 意味着“所有”（所有文件、所有主机或所有命令），NOPASSWD 意味着不用密码。
- 4) 通过修改用户访问，将普通用户的访问权限修改为同 root 一样，则可以给普通用户分配特权。

下面是一个 sudoers 文件的例子：

```
#sudoers files

#User alias specification

User_Alias ADMIN=ted1:POWERUSER=globus,ted2

#user privilege specification

ADMIN ALL=ALL

POWERUSER ALL=ALL,!/bin/su
```

其中：

- 1) User_Alias ADMIN=ted1:POWERUSER=globus,ted2
定义了两个别名 ADMIN 和 POWERUSER
- 2) ADMIN ALL=ALL
说明在所有主机上，ADMIN 用户都可以以 root 身份执行所有命令
- 3) POWERUSER ALL=ALL,!/bin/su
给 POWERUSER 用户除了运行 su 命令外等同 ADMIN 的权限

题目三 基于 OpenEuler 操作系统的应用开发

实验内容

1、安装 JDK 软件包

(1) 执行 `dnf list installed | grep jdk` 查询 JDK 软件是否已安装。

查看命令打印信息，若打印信息中包含“jdk”，表示该软件已经安装了，则不需要再安装。若无任何打印信息，则表示该软件未安装。

(2) 清除缓存。

```
$ dnf clean all
```

(3) 创建缓存。

```
$ dnf makecache
```

(4) 查询可安装的 JDK 软件包。

```
$ dnf search jdk | grep jdk
```

(5) 查看命令打印信息，选择安装 `java-x.x.x-openjdk-devel.aarch64` 软件包。其中 `x.x.x` 为版本号，同时支持 `openjdk 1.8`、`openjdk 11` 和 `openjdk latest` 版本。在 `root` 权限下安装 JDK 软件包，以安装 `java-1.8.0-openjdk-devel` 软件包为例。

```
# dnf install java-1.8.0-openjdk-devel.aarch64
```

(6) 查询 JDK 软件版本。

```
$ java -version
```

(7) 查看打印信息，若打印信息中包括“`openjdk version "1.8.0_232"`”信息，表示已正确安装，其中 `1.8.0_232` 为版本号。

2、下载并运行 IntelliJ IDEA

对于小型的 Java 程序，可以直接参考“使用 JDK 编译”章节得到可运行 Java 应用。但是对于大中型 Java 应用，这种方式已经无法满足开发者的需求。

可以下载使用 IntelliJ IDEA 了。鉴于最新版的 IntelliJ IDEA 和 openEuler 系统在部分功能上有兼容性问题，从此以下链接下载 2018 版本 linux 压缩包：

<https://www.jetbrains.com/idea/download/other.html>

下载好后把压缩包移到您想要安装该软件的目录，对压缩包进行解压：

```
$ tar xf ideaIC-2018.3.tar.gz
```

解压后切换到 IntelliJ IDEA 的目录下并运行。

```
$ cd ./idea-IC-183.4284.148
```

```
$ bin/idea.sh &
```

3、基于上述搭建的开发环境，选择下面编程实现的题目进行编程实现。

题目四 Docker 技术的应用

实验内容

Docker 是一个开源的 Linux 容器引擎项目，用以实现应用的快速打包、部署和交付。Docker 的英文本意是码头工人，码头工人的工作就是将商品打包到 container(集装箱)并且搬运 container、装载 container。对应到 Linux 中，Docker 就是将 app 打包到 container，通过 container 实现 app 在各种平台上的部署、运行。Docker 通过 Linux Container 技术将 app 变成一个标准化的、可移植的、自管理的组件，从而实现应用的“一次构建，到处运行”。Docker 技术特点就是：应用快速发布、部署简单、管理方便，应用密度更高。

要求将编程实现的程序打包成 docker 的容器。

1、docker 命令

(1) docker login 登录到 Docker 镜像仓库，默认为官方仓库

docker logout 登出 Docker 镜像仓库，默认为官方仓库

登录:docker login -u “用户名” -p “密码”

登出:docker logout

(2) 用法: docker pull [OPTIONS] NAME[:TAG]

功能: 从一个镜像库（官方的或私有的）中拉取一个镜像

-a, --all-tags=false 下载一个镜像仓库的所有镜像

docker pull java

docker pull -a java

(3) Push用法: docker push NAME[:TAG]

将本地的镜像上传到镜像仓库，要先登录到镜像仓库

功能: 将一个 image 推送到镜像库中

实例:

上传本地镜像 java 镜像仓库中

docker login

docker push java

Search

docker search:从 docker hub 查找镜像

用法: docker search [OPTIONS] TERM

功能: 在镜像库中查找特定的镜像

--automated=false 显示自动构建的 image

--no-trunc=false 不对输出进行删减

-s, --stars=0 只显示特定星级以上的 image

2、容器生命周期管理

(1) run 用法: docker run [OPTIONS] IMAGE [COMMAND] [ARG...]

功能: 该命令将由指定的 image（如果指定的 IMAGE 不存在，则从官方镜像库中下载一个镜像）创建一个容器，并启动这个容器，并在容器中执行指定的命令。该命令集成了 docker create 命令、docker start 命令、docker exec 命令。

选项:（该命令的选项与 docker create 命令的选项一样，请参考 docker create 命令选项，仅仅多了以下两个选项）

--rm=false 设置容器退出时自动删除容器

-v 挂载本地目录或匿名卷到容器内。注意：当将本地目录以带有 selinux 的安全标签的方式挂载到容器内的同时，尽量不要同时做该本地目录的增删操作，否则该安全标签可能不生效

--sig-proxy=true 发往进程信号的代理, SIGCHLD, SIGSTOP, SIGKILL 不使用代理
示例:

使用 busybox 镜像运行一个容器, 在容器启动后执行 /bin/sh

实例: `docker run -it busybox /bin/sh`

(2) docker start/stop/restart

停止运行中的容器 busybox: `Docker stop busybox`

启动已被停止的容器 busybox: `Docker start busybox`

重启容器 busybox: `Docker restart busybox`

(3) docker 删除一个或多个容器

用法: `docker rm [OPTIONS] CONTAINER [CONTAINER...]`

功能: 删除一个或多个容器

-f, --force=false 强制删除运行中的容器

-l, --link=false Remove the specified link and not the underlying container

-v, --volumes=false Remove the volumes associated with the container

实例: `docker rm busybox`

(4) docker create: 创建一个新的容器但不启动它, 拉取镜像后创建一个新的容器, 创建之后用 `docker start` 命令启动:

实例: `Docker create -it --name=busybox busybox /bin/bash`

(5) 在运行的容器中执行命令:

`docker exec [OPTIONS] CONTAINER COMMAND [ARG...]`

-d: 分离模式: 在后台运行

-i: 即使没有附加也保持 STDIN 打开

-t: 分配一个伪终端

实例: `Docker exec -it busybox /bin/sh`

当然 busybox 也可以使用容器 ID 代替

3、本地镜像管理

(1) images: 列出本地镜像

`Docker images [OPTIONS] [REPOSITORY] [:TAG]`

-a: 列出本地所有的镜像(含中间映像层)

-f: 显示满足条件的镜像

-q: 只显示镜像 ID

--no-trunc: 显示完整的镜像信息

实例: `docker images`

(2) rmi 命令: 删除本地镜像

`Docker rmi [OPTIONS] IMAGE [IMAGE...]`

-f: 强制删除

--no-prune: 不删除该镜像的过程镜像, 默认移除

实例: `Docker rmi -f busybox`

(3) docker tag: 标记本地镜像, 将其归入某一仓库

`docker tag [OPTIONS] IMAGE[:TAG] [REGISTRYHOST/][USERNAME/]NAME[:TAG]`

把镜像 busybox 标记为 runoob/busybox:v3 镜像

实例: `docker tag busybox runoob/busybox:v3`

(4) docker build [OPTIONS] PATH | URL | -

功能: 使用指定路径中的 Dockerfile 生成构建一个新的 image

--force-rm=false 即使没有构建成功也删除构建过程中生成的容器

--no-cache=false 构建 image 的过程中不使用缓存
-q, --quiet=false 禁止构建过程中产生的冗余信息
--pull 构建时总是尝试获取
--rm=true 禁止构建过程中产生的冗余信息

实例: Docker build -t busybox:latest

通过以下命令可以看到这个生成 image

实例: Docker image | grep busybox

(5) History

用法: docker history [OPTIONS] IMAGE

功能: 显示一个 image 的变化历史

-H, --human=true

--no-trunc=false 不对输出进行删减

-q, --quiet=false 只显示 ID

实例: Docker history busybox:test

(6) Save

用法: docker save [OPTIONS] IMAGE [IMAGE...]

功能: 保存一个 image 到一个 tar 包, 输出默认是到 STDOUT

-o, --output="" 输出到文件中而不是 STDOUT

实例: docker save -o nginx.tar nginx:latest

(7) Load 导入使用 docker save 命令导出的镜像

用法: docker load [OPTIONS]

功能: 把 docker save 出来的 tar 包重新加载一个镜像。与 docker save 相对应。

-i, --input=""

实例: docker load -i busybox.tar

4、容器操作

(1) ps

用法: docker ps [OPTIONS]

功能: 根据不同的选项列出不同状态的容器, 在不加任何选项的情况下, 将列出正在运行的容器

-a, --all=false 显示所用的容器

-f, --filter=[] 筛选值, 可用的筛选值有: exited=<int>容器的退出码

status=(restarting|running|paused|exited)容器的状态码 (e.g. -f status=running, 列出正在运行的容器)

-l, --latest=false 列出最近创建的一个容器

-n=-1 列出最近 n 此创建的容器

--no-trunc=false 将 64 位的容器 ID 全部显示出来, 默认显示 12 位容器的 ID

-q, --quiet=false 显示容器的 ID

-s, --size=false 显示容器的大小

实例:

1) 列出正在运行的容器

docker ps

2) 列出所有的容器, 包括运行中的容器以及停止的容器

docker ps -a

(2) inspect 命令

用法: docker inspect [OPTIONS] CONTAINER|IMAGE [CONTAINER|IMAGE...]

功能：返回一个容器或者镜像的底层信息
-f, --format="" 按照给定的格式输出信息
-s, --size 若查询类型为容器，显示该容器的总体文件大小
--type 返回指定类型的 JSON 格式
-t, --time=120 超时时间的秒数，若在该时间内 docker inspect 未执行成功，则停止等待并立即报错。默认为 120 秒。

(3) Wait

docker wait:阻塞运行直到容器停止，然后打印出它退出代码

docker wait [OPTIONS] CONTAINER [CONTAINER...]

实例：docker wait CONTAINER

5、容器管理案例

(1) 拉取 busybox 镜像

docker pull busybox

(2) 运行一个简单的应用

docker run busybox /bin/echo "Hello world"

(3) 查看刚才创建的容器

docker ps -l

(4) 创建一个交互式的容器

docker run -it busybox /bin/sh

(5) 后台运行程序

docker run -d --name=container1 busybox /bin/sh -c "while true;do echo hello world;sleep 1;done"

(6) 使用 docker logs 查看容器运行的输出

docker logs container1

6、运行 Java 程序

(1) 创建 Java 文件目录

(2) 创建 Dockerfile 文件，并在文件中加入 Java 编辑命令：

FROM java:7

COPY . /usr/src/javaapp

WORKDIR /usr/src/javaapp

RUN javac HelloWorld.java

CMD ["java", "HelloWorld"]

(3) 创建并编辑 Java 文件

(4) 构建 Java-image 镜像：

#docker build -t java-image .

(5) 运行镜像：

#docker run -it --rm --name java-container java-image

编程设计类题目（从下列几个题目中选）

两种选择：①题目一和题目二任选一个，且题目三和题目四任选一个（2个题目作为一组）
②题目五和题目六任选一个

题目一 进程同步与互斥

一、课程设计目的

OpenEuler 操作系统基本是多任务操作系统，即同时有大量可调度实体在运行。在多任务操作系统中，同时运行的多个任务可能：

- 都需要访问/使用同一种资源；
- 多个任务之间有依赖关系，某个任务的运行依赖于另一个任务。

掌握基本的同步与互斥算法，掌握进程并发执行的原理，及其所引起的同步、互斥问题的方法。

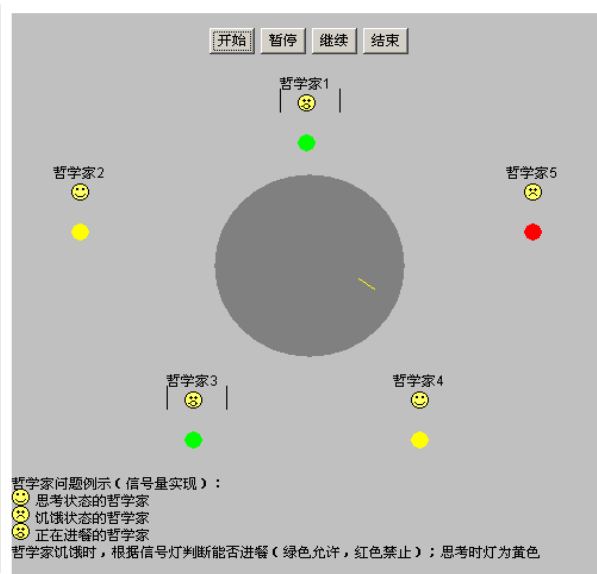
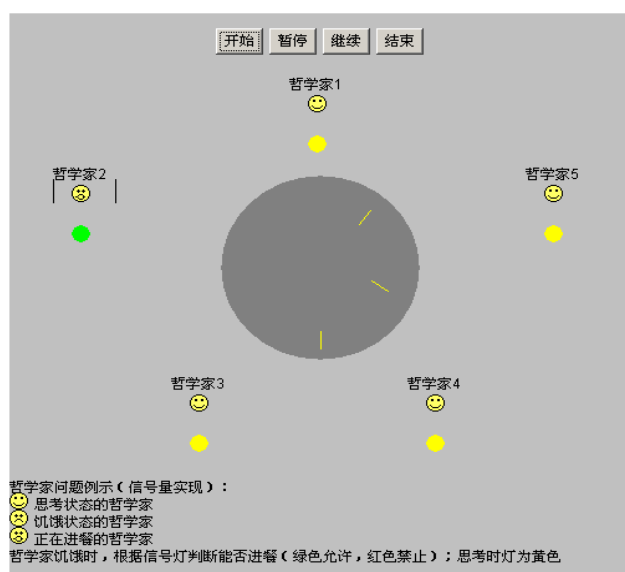
二、课程设计内容

在 OpenEuler 操作系统中，使用高级编程语言（如 Java 等），自己编写信号量和 wait、signal 操作的模拟程序，然后用它们解决不死锁的哲学家问题或者读者-写者问题。

三、要求及提示

1、解决不死锁的哲学家问题，要求把哲学家们的活动过程用文字或图形可视化形式表示出来。

提示：首先设置一个“PCB”数组或队列，其中一个字段表示“阻塞原因兼阻塞标志”，本实验中，该数组有 5 个元素表示 5 个哲学家即可。它们随机提出申请以及进行“思考”“吃”的行为，再设一个“筷子”数组。还需要设置哪些数据结构以及需要哪些字段自己考虑。示例图如下，仅供参考。



2、解决读者-写者问题，仅要求解决读者优先的情况。

提示：创建一个控制台进程，此进程包含 n 个线程。用这 n 个线程来表示 n 个读者或写者。每个线程按相应测试数据文件的要求进行读写操作。

读者-写者问题的读写操作限制：

- 1)写-写互斥，即不能有两个写者同时进行写操作。
- 2)读-写互斥，即不能同时有一个线程在读，而另一个线程在写。
- 3)读-读允许，即可以有一个或多个读者在读。

读者优先的附加限制：如果一个读者申请进行读操作时已有另一个读者正在进行读操作，则该读者可直接开始读操作。但任何写者必须等到没有读者和写者时才能开始写操作。

运行结果显示要求：在每个线程创建、发出读写操作申请、开始读写操作和结束读写操作时分别显示一行提示信息，以确定所有处理都遵守相应的读写操作限制。

测试数据文件包括 n 行测试数据，分别描述创建的 n 个线程是读者还是写者，以及读写操作的开始时间和持续时间。每行测试数据包括四个字段，各个字段间用空格分隔。第一字段为一个正整数，表示线程序号。第二字段表示相应线程角色，R 表示读者，W 表示写者。第三字段为一个正数，表示读写操作的开始时间，线程创建后，延迟相应时间(单位为秒)后发出对共享资源的读写申请。第四字段为一个正数，表示读写操作的持续时间。当线程读写申请成功后，开始对共享资源的读写操作，该操作持续相应时间后结束，并释放共享资源。测试数据随机生成，为简化起见，假设每个线程只执行一次读或写操作，之后运行结束。

下面是一个测试数据文件的例子：

线程序号	角色	何时开始读写	读写持续时间
1	R	3	5
2	R	4	5
3	W	5	2
4	R	10	3
5	W	11	3
6	R	13	4
.....			

可能用到的 API 函数：

CreateThread()	在调用进程的地址空间上创建一个线程
ExitThread()	用于结束当前线程
Sleep()	可在指定的时间内挂起当前线程
CreateMutex()	创建一个互斥对象，返回对象句柄
OpenMutex()	打开并返回一个已存在的互斥对象句柄，用于后续访问
ReleaseMutex()	释放对互斥对象的占用，使之成为可用
WaitForSingleObject()	可在指定的时间内等待指定对象为可用状态

InitializeCriticalSection()	初始化临界区对象
EnterCriticalSection()	等待指定临界区对象的所有权
LeaveCriticalSection()	释放指定临界区对象的所有权
CreateSemaphore()	创建一个信号量对象
ReleaseSemaphore()	将所指信号量加上指定大小的一个量

题目二 页面置换算法的模拟实现及命中率对比

一、课程设计目的

OpenEuler 操作系统在地址映射过程中，若在页面中发现所要访问的页面不在内存中，则产生缺页中断。当发生缺页中断时，如果操作系统内存中没有空闲页面，则操作系统必须在内存选择一个页面将其移出内存，以便为即将调入的页面让出空间。

通过请求页式管理方式中页面置换算法的模拟设计，了解虚拟存储技术的特点，掌握请求页式存储管理中的页面置换算法。

二、课程设计内容

在 OpenEuler 操作系统中，使用高级编程语言模拟实现最佳置换法（OPT）、先进先出法(FIFO)和最近最少使用法(LRU)，并计算每种算法的缺页率。

三、要求及提示

1、首先用随机数生成函数产生一个“指令将要访问的地址序列”，然后将地址序列变换成相应的页地址流（即页访问序列），再计算不同算法下的命中率。

2、通过随机数产生一个地址序列，共产生 400 条。其中 50%的地址访问是顺序执行的，另外 50%就是非顺序执行。且地址在前半部地址空间和后半部地址空间均匀分布。具体产生方法如下：

- 1) 在前半部地址空间，即[0, 199]中随机选一数 m ，记录到地址流数组中（这是非顺序执行）；
- 2) 接着“顺序执行一条指令”，即执行地址为 $m+1$ 的指令，把 $m+1$ 记录下来；
- 3) 在后半部地址空间，[200, 399]中随机选一数 m' ，作为新指令地址；
- 4) 顺序执行一条指令，其地址为 $m'+1$ ；
- 5) 重复步骤 1~4，直到产生 400 个指令地址。

3、将指令地址流变换成页地址（页号）流，简化假设为：

- 1) 页面大小为 1K（这里 K 只是表示一个单位，不必是 1024B）；
- 2) 用户虚存容量为 40K；
- 3) 用户内存容量为 4 个页框到 40 个页框；
- 4) 用户虚存中，每 K 存放 10 条指令，所以那 400 条指令访问地址所对应的页地址（页号）流为：指令访问地址为[0, 9]的地址为第 0 页；指令访问地址为[10, 19]的地址为

第 1 页；……。按这种方式，把 400 条指令组织进“40 页”，并将“要访问的页号序列”记录到页地址流数组中。

4、循环运行，使用户内存容量从 4 页框到 40 页框。计算每个内存容量下不同页面置换算法的命中率。输出结果可以为：

页框数	OPT 缺页率	FIFO 缺页率	LRU 缺页率
[4]	OPT: 0.5566	FIFO: 0.4455	LRU: 0.5500
[5]	OPT: 0.6644	FIFO: 0.5544	LRU: 0.5588
.....			
.....			
[39]	OPT: 0.9000	FIFO: 0.9000	LRU: 0.9000
[40]	OPT: 1.0000	FIFO: 1.0000	LRU: 1.0000

注 1：在某一次实验中，可能 FIFO 比 LRU 性能更好，但足够多次的实验表明 LRU 的平均性能比 FIFO 更好。

注 2：计算缺页率时，以页框填满之前和之后的总缺页次数计算。

题目三 单处理器系统的进程调度

一、课程设计目的

在 OpenEuler 操作系统中，用户进程数一般都多于处理机数、这将导致它们互相争夺处理机。另外，系统进程也同样需要使用处理机。这就要求进程调度程序按一定的策略，动态地把处理机分配给处于就绪队列中的某一个进程，以使之执行。

1. 加深对进程概念的理解，明确进程和程序的区别。
2. 深入了解系统如何组织进程、创建进程。
3. 进一步认识如何实现处理器调度。

二、课程设计内容

在 OpenEuler 操作系统中，使用高级编程语言编写程序完成单处理器系统中的进程调度，要求实现时间片轮转、优先数、最短进程优先和最短剩余时间优先四种调度算法。实验具体包括：首先确定进程控制块的内容，进程控制块的组成方式；然后完成进程创建原语和进程调度原语；最后编写主函数对所做工作进行测试。

模拟程序只对你所设置的“虚拟 PCB”进行相应的调度模拟操作，即每发生“调度”时，显示出当前运行进程的“进程标识符”、“优先数”、“剩余运行时间”等，而不需要对系统中真正的 PCB 等数据进行修改。

三、要求及提示

要求能够动态地随机生成新进程添加到就绪队列中。

主要考虑三个问题：如何组织进程、如何创建进程和如何实现处理器调度。

1、组织进程

考虑如何组织进程，首先要设定进程控制块的内容。进程控制块 PCB 记录各个进程执行时的情况。不同的操作系统，进程控制块记录的信息内容不一样。操作系统功能越强，软件也越庞大，进程控制块的内容也就越多，这里只使用必不可少的信息。一般操作系统中，无论进程控制块中信息量多少，信息都可以大致分为以下四类：

（1）标识信息

每个进程都要有一个唯一的标识符，用来标识进程的存在和区别于其他进程。这个标识符是不可少的，可以用符号或编号实现，它必须是操作系统分配的。例如采用编号方式，也就是为每个进程依次分配一个不相同的正整数。

（2）说明信息

用于记录进程的基本情况，例如进程的状态、等待原因、进程程序存放位置、进程数据存放位置等等。实验中，因为进程没有数据和程序，仅使用模拟的进程控制块，所以这部分内容仅包含进程状态。

进程状态可假设只有就绪、运行、终止三种。如果要设置“等待（阻塞）”状态，需要随机生成“阻塞时间”，阻塞时间到时转为就绪态。

（3）现场信息

现场信息记录各个寄存器的内容。当进程由于某种原因让出处理器时，需要将现场信息记录在进程控制块中，当进行进程调度时，从选中进程的进程控制块中读取现场信息进行现场恢复。现场信息就是处理器的相关寄存器内容，包括通用寄存器、程序计数器和程序状态字寄存器等。**在本实验中，本部分可忽略。**

（4）管理信息

管理信息记录进程管理和调度的信息。例如进程优先数、进程队列指针等。

另外，本实验为了模拟进程的不同运行时间，再添加一个“剩余运行时间”属性。

因此可将进程控制块结构定义如下：

```
struct pcb
{ int name;           //进程标识符
  int status;         //进程状态
  int pri;            //进程优先数
  int time;           //剩余运行时间，以时间片为单位，当减至 0 时该进程终止
  int next;           //下一个进程控制块的位置
}
```

确定进程控制块内容后，要考虑的就是如何将进程控制块组织在一起。多道程序设计系统中，往往同时创建多个进程。在单处理器的情况下，每次只能有一个进程处于运行态，其它的进程处于就绪状态或者等待状态。为了便于管理，通常把处于相同状态的进程的进程控制块链接在一起组成就绪队列和等待队列。

由于实验模拟的是进程调度，没有对等待队列的操作，所以实验中只有一个指向正在运行进程的进程控制块指针和一个就绪进程的进程控制块队列指针。操作系统实现中，系统往往在主存中划分出一个连续的专门区域存放系统的进程控制块，实验中应该用数组模拟这个专门的进程控制块区域，定义如下：

```
#define n 10           //假定系统允许进程个数为 n
struct pcb pcbarea[n]; //模拟进程控制块区域的数组
```

这样，进程控制块的链表实际上是数据结构中使用的静态链表。实验中，进程控制块队列采用单向不循环静态链表。为了管理空闲进程控制块，还应该将空闲控制块链接成一个队列。

进程调度其实就是一个排队的过程，不同的算法区别在于按照什么样的次序将就绪队列里面的进程进行排序，比如时间片轮转调度算法，是将进程控制块按照进入就绪队列的先后次序排队。关于就绪队列的操作就是从队头摘下一个进程控制块和从队尾挂入一个进程控制块。因此为就绪队列定义两个指针，一个头指针，指向就绪队列的第一个进程控制块；一个尾指针，指向就绪队列的最后一个进程控制块。

实验中指向运行进程的进程控制块指针、就绪队列指针和空闲进程控制块队列指针定义如下：

```
int run;           //定义指向正在运行进程的进程控制块的指针
struct
{
    int head;
    int tail;      //定义指向就绪队列的头指针 head 和尾指针 tail
}ready;
int pfree;        //定义指向空闲进程控制块队列的指针
```

2、创建进程

进程创建是一个原语，因此在实验中应该用一个函数实现，进程创建的过程应该包括：

（1）申请进程控制块

进程控制块的数量是有限的，如果没有空闲进程控制块，则进程不能创建，如果申请成功才可以执行第二步。

（2）填写进程控制块

将该进程信息写入进程控制块内。进程标识符应该随机生成并且是唯一，优先数和剩余运行时间随机生成，刚刚创建的进程为就绪态，然后转去执行第三步。

（3）挂入就绪队列

如果原来就绪队列不为空，则将该进程挂入就绪队列尾部，并修改就绪队列尾部指针；如果原来就绪队列为空，则就将就绪队列的头指针、尾指针均指向该进程控制块，进程创建完成。

多道程序设计的系统中，处于就绪状态的进程往往是多个，它们都要求占用处理器，可是单处理器系统的处理器只有一个，进程调度就是解决这个处理器竞争问题的。进程调度的任务就是按照某种算法从就绪进程队列中选择一个进程，让它占有处理器。因此进程调度程序就应该包括两部分，一部分是在进程就绪队列中选择一个进程，并将其进程控制块从进程就绪队列中摘下来，另一部分工作就是分配处理器给选中的进程，也就是将指向正在运行进程的进程控制块指针指向该进程的进程控制块，并将该进程的进程控制块信息写入处理器的各个寄存器中。

提醒注意的是：在实际的系统中，当一个进程被选中运行时，必须恢复进程的现场，让它占有处理器运行，直到出现等待事件或运行结束。在本实验里省去了这些工作。

以时间片轮转调度算法为例说明如何挂入就绪队列。时间片轮转调度算法让就绪进程按就绪的先后次序排成队列，每次总是选择就绪队列中的第一个进程占有处理器，但是规定只能使用一个“时间片”。时间片就是规定进程一次使用处理器的最长时间。实验中采用每个进程都使用相同的不变时间片。每被调度 1 次，将其剩余运行时间-1。

进程运行一次后，若剩余运行时间不等于 0，则再将它加入就绪队列尾；若剩余运行时间等于 0，则把它的状态修改成“终止”，且退出队列。

若就绪进程队列不为空，则重复调度，直到所有进程都“终止”。

3、调度

完成上述功能后，编写主函数进行测试：首先建立一个就绪队列，随机生成信息建立若干个进程；然后进行进程调度；将正在运行进程指针指向的进程控制块的内容以及调度一次后进程队列的现状输出，查看结果。题目要求模拟实现四种调度方法，要求和提示如下：

(1) 时间片轮转调度

时间片轮转调度的要求和提示已经在上述过程中进行了说明，请参看。

(2) 优先数调度

要求动态改变优先数，假设大数代表高优先级，进程每运行一次优先数就减“1”，即被调度时执行：优先数-1，剩余运行时间-1，来模拟进程的一次运行。

进程运行一次后，若剩余运行时间不等于 0，则再将它加入队列（按优先数大小插入，且置队首标志）；若剩余运行时间等于 0，则把它的状态修改成“终止”，且退出队列。

若就绪进程队列不为空，则重复调度，直到所有进程都“终止”。

(3) 最短进程优先

按照进程执行时间的长短进行排队，优先调度短进程，因为该调度不抢占，因此调度到的进程就可以运行完，其状态修改成“终止”，且退出队列。

若就绪进程队列不为空，则重复调度，直到所有进程都“终止”。

(4) 最短剩余时间优先

最短进程优先的抢占版本，比较当前进程的剩余运行时间和新到达进程的预计运行时间，短者优先执行。进程运行一次后，若剩余运行时间不等于 0，则再将它加入队列（按剩余时间长短插入，且置队首标志）；若剩余运行时间等于 0，则把它的状态修改成“终止”，且退出队列。

若就绪进程队列不为空，则重复调度，直到所有进程都“终止”。

题目四 磁盘调度算法的模拟实现及对比

一、课程设计目的

对磁盘发起 I/O 操作的时间主要由寻道时间和旋转时间决定，磁盘在不同时刻访问不同扇区的时间成本是不同的。当有多个 I/O 请求发出时,磁盘先服务哪个请求会极大地影响 I/O 性能。比如，当前磁头在内侧磁道上，有三个 I/O 请求分别访问内、中、外侧磁道上的扇区。显然，这种情况下自内向外处理请求比从外向内处理请求更加高效。

在 OpenEuler 操作系统中，为了获得更高的 I/O 性能，磁盘必须对 I/O 请求进行调度。这些调度的主要依据是当前磁头的位置和要访问的扇区的位置信息。通过磁盘调度算法的模拟设计，了解磁盘调度的特点。

二、课程设计内容

在 OpenEuler 操作系统中，使用高级编程语言模拟实现 FCFS、SSTF、电梯 LOOK、C-SCAN 算法，并计算及比较磁头移动道数。

三、要求及提示

1、首先假设磁盘磁道数为 1500，磁头初始位置可任意设置。

2、用随机数生成函数产生“磁道号”序列（即磁盘请求的位置），共产生 400 个。其中 50%位于 0~499，25%分布在 500~999，25%分布在 1000~1499。具体的产生方法可参考“题目一 页面置换算法的模拟实现及命中率对比”。

3、计算及比较每种磁盘调度算法下的磁头移动道数。

注：本题目要求给出图形可视化界面，并且能够动态模拟每个算法的调度过程，可采用从上一个请求到下一个请求进行连线的方式。

题目五 模拟磁盘文件系统实现

一、课程设计目的

文件系统是 OpenEuler 操作系统中藉以组织、存储和命名文件的结构。磁盘或分区和它所包括的文件系统的不同是很重要的，大部分应用程序都基于文件系统进行操作，在不同种文件系统上是不能工作的。了解磁盘文件系统的结构、功能和实现。并可练习合作完成系统的团队精神 and 提高程序设计能力。

二、小组人数

5 人一组共同完成模拟磁盘文件系统的实现。

三、编程语言

在 OpenEuler 操作系统中，使用高级编程语言（如 Java），来模拟并行发生的行为。要求图形界面。

四、课程设计内容

设计一个简单的文件系统，用文件模拟磁盘，用数组模拟缓冲区，要求：

- (1) 支持多级目录结构，支持文件的绝对读路径；
- (2) 文件的逻辑结构采用流式结构，物理结构采用链接结构中的显式链接方式；
- (3) 采用文件分配表 FAT；
- (4) 实现的命令包括建立目录、列目录、删除空目录、建立文件、删除文件、显示文件内容、打开文件、读文件、写文件、关闭文件、改变文件属性。
- (5) 最后编写主函数对所作工作进行测试。

五、课程设计具体内容和要求

为了正确地实现文件的存取，文件系统设计了一组与存取文件有关的功能模块，用户可以用“访管指令”调用这些功能模块，以实现文件的存取要求。我们把文件系统设计的这一组功能模块称为“文件操作”，实验就是要模拟实现一些文件操作。文件操作不是独立的，它和文件系统的其它部分密切相关，若要实现文件操作就离不开文件的目录结构、文件的组织结构和磁盘空间的管理。因此，这个实验虽然是文件操作的模拟实现，但是还必须模拟一部分文件的组织结构、目录结构和磁盘空间管理的实现。

(1) 文件的组织结构

文件的逻辑结构有两种形式：流式文件和记录式文件。实验中只支持流式文件，采用称为显式链接的物理文件结构，把磁盘中每一块的指针部分提出来，组织在一起，形成文件分配表（FAT）。文件分配表的作用不仅如此，其它的作用下面将提到。

磁盘有多少块，文件分配表就有多少项，若某文件的一个磁盘块号为 i ，则这个文件的下一个磁盘块号应该记录在文件分配表的第 i 项。例如，某系统文件分配表的前几项值如下图所示。某个

文件的起始盘块号为 3，则该文件的磁盘块号依次为 3、4、9、12、13。

项	0	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15	
内容	-1	-1	-1	4	9	0	7	8	-1	12	11	-1	13	-1	0	0	

(2) 磁盘空间的管理

首先要模拟一个磁盘。因为是实验，不使用真正的磁盘，所以实验中用一个文件模拟一个小磁盘。假设模拟磁盘有 128 个物理块，每个物理块大小为 64 字节。盘块的块号从 0 编起。

将前面所讲的文件分配表放在磁盘的开始处，因为盘块有 128 块，所以文件分配表有 128 项，每项占用一个字节。这样文件分配表占用了磁盘的 0 块和 1 块，这两块就不能作其它用处。若一个盘块是某个文件的最后一块，填写“-1”表示文件结束。

文件的建立和删除就需要对磁盘的空间进行分配和回收，所以要建立一定的数据表格来记录磁盘的使用情况。用文件分配表的第 i 项表示第 i 个盘块的使用情况。磁盘的第 0 块一定会被系统数据占用，所以任何一个文件的某个盘块块号都不可能是“0”，因而可以用“0”表示磁盘盘块空闲，若这个盘块已经分配出去，即是某个文件的一块，由上面我们知道文件分配表中对应项记录的是文件下一块的块号或结束标志都不是“0”。这样非“0”值表示盘块已分。像前图那张文件分配表中，块号为 5、14 和 15 的盘块是空闲的，其余是已分配的。在文件分配表中可以用一个超过盘块编号的正整数表示文件结束，在此实验中采用 255 代替-1 表示文件结束。

如果磁盘中某些部分损坏，只要不是系统区（引导扇区、文件分配表或根目录等），不分配那些坏的盘块，磁盘可以继续使用。在文件分配表对应坏盘块的项不能是“0”，一般需要特定的数值表示（这个数值应该是盘块编号以外并且不是结束标志的数值，例如，实验中采用 128~254 之间的某个数值）。假设实验中模拟磁盘的第 23 块和第 49 块已经损坏，不能使用，则在文件分配表的第 23 项和第 49 项写入“254”表示该盘块损坏不能使用。

由于磁盘分配时，有时不能预定文件的大小，例如建立文件时并不知道文件的大小。因而磁盘的分配有时是一块一块申请的。磁盘空间回收时，整个文件删除时回收很多，但有时文件修改时可能会删除某些内容，造成归还磁盘块，这时是一块一块回收的。注意：分配一个磁盘块时，不应该从文件分配表第一项查起，因为磁盘中最开始的几块为系统数据区，可假定系统区域占用了 x 个盘块，则应该从第 x 块（0~ $x-1$ 块为系统区域）开始查询。回收一个磁盘块很简单，比如回收磁盘块的块号为 x ，只要找到文件分配表中第 x 项，将第 x 项的值改为 0 即可。

(3) 目录结构

文件目录是用于检索文件的，它是文件系统实现按名存取的主要手段。文件目录由若干目录项组成，每一个目录记录一个文件的有关信息。一般的说，目录项应该包括如下内容：

①有关文件的控制信息。例如，用户名、文件名。文件类型、文件属性。实验模拟个人计算机上的文件操作，这部分内容仅包括文件名、文件类型和文件属性；

②有关文件结构的信息。例如，文件的逻辑结构、文件的物理结构、记录个数、文件在存储介质的位置等。实验中，仅仅支持流式文件，不支持记录式文件，所以这部分内容仅仅包括文件在存储介质的位置（分给文件第一个盘块的块号，即起始盘块号）、文件的长度；

③有关文件管理的信息。例如，文件的建立日期、文件被修改的日期、文件保留期限和记帐信息等。实验中为了简单起见，这部分内容都不采用。

因此，实验中文件的目录项包括：文件名、文件类型、文件属性、文件的起始盘块号、文件的长度，每个目录项占用 8 个字节，具体结构如下所示：

文件名：3 个字节（实验中合法文件名仅可以使用字母、数字和除 “\$”、“.”、“/” 以外的字符，第一个字节的值为 “\$” 时表示该目录为空目录项，文件名和类型名之间用 “.” 分割，用 “/” 作为路径名中目录间分隔符）；

文件类型名：2 个字节；

文件属性：1 个字节；

起始盘块号：1 个字节；

文件长度：1 个字节（为了实验的简单，假设文件长度单位为盘块）

有了文件目录后，当用户要求使用某个文件时，文件系统可以顺序查找目录项，并比较文件名，就可以找到指定文件的目录项，根据目录行中有关内容核对使用权限、并读出文件供用户使用。因此文件目录的组织和管理要便于检索和防止冲突。

在操作系统中目录就有根目录和子目录两种。除了文件需要登记形成目录外，还要登记子目录的情况。实验中，根目录固定位置、固定大小（可以登记有限个文件或子目录项），子目录像文件一样，可使用任何一个空闲磁盘块。为了实验简单，实验中根目录占用一个盘块，子目录的长度不采用可以任意长的方法，而是采用定长的方法，每个子目录的长度也是一个盘块，只能放 8 个目录项。文件和目录的登记项是混在一起的，登记项的结构应该和文件目录一样，每个目录项占用 8 个字节，结构如下：

目录名：3 个字节（实验中合法目录名仅可以使用字母、数字和除 “\$”、“.”、“/” 以外的字符，第一个字节的值为 “\$” 时表示该目录为空目录项）；

保留 2 字节未使用（在实验中填写空格）；

目录属性：1 个字节；

起始盘块号：1 个字节；

保留 1 字节未使用（在实验中填写 “0”）。

在目录登记项中，系统为目录名后 2 个字节（对应文件类型名位置）填写空格，目录起始盘块号后 1 字节（对应文件长度位置）填写 0。目录属性和文件属性占用同一个字节，为了区别目录和文件，该字节每一位代表不同的含义（为 “1” 表示 “是”，为 “0” 表示否），如下图所示，第 0 位表示文件为只读文件，第 1 位表示文件为系统文件，第 2 位表示文件为可读、可写的普通文件，第 3 位表示该登记项不是文件的登记项，而是目录的登记项，其余几位闲置未用。如该字节为 8（00001000），表示该目录是一个目录的登记项，该字节为 3（00000011），表示该目录是一个只读系统文件的登记项，该字节为 4（000000100），表示该目录是一个可读可写的普通文件。

第 7 位	第 6 位	第 5 位	第 4 位	第 3 位	第 2 位	第 1 位	第 0 位
-------	-------	-------	-------	-------	-------	-------	-------

未使用	未使用	未使用	未使用	目录属性	普通文件	系统文件	只读文件
-----	-----	-----	-----	------	------	------	------

目录检索的方法常用的是顺序检索，根据绝对路径名查找文件的方法一般如下：先找到根目录的起始盘块，一般根目录位置是固定的，实验中就是模拟磁盘的第 2 块，将该盘块读出；取出路径名中根目录后的目录名或文件名，和根目录中目录项依次比较，比较完一块，再根据文件分配表找到下一块，再读入比较，直到找到名字一致的目录项或根目录登记项均已查完为止；若没有找到，则查找失败，结束；若查找的是文件，结束；若查找的是目录，从查找到的目录项中，取出目录的起始盘块号，读入此盘块，然后用上述相同的查找方法继续查找，直到找到该文件（或目录）或查找失败结束。

查找文件除了绝对路径名外，还可以使用相对路径名。相对路径名是从当前目录出发到指定文件的路径。如果文件（或目录）在当前目录下，使用相对路径名查找速度比较快。和绝对路径的查找方法一样，只是查找的起点是当前目录，不是根目录。实验中只使用绝对路径名。

（4） 文件操作

确定文件组织结构、目录结构和磁盘空间管理的方法后，就可以模拟文件操作的实现。实验中文件操作包括建立文件、打开文件、关闭文件、读文件、写文件、删除文件、显示文件内容和改变文件属性，目录命令包括建立目录、显示目录内容和删除空目录。在实验中没有程序调用这些命令，为了看到它们的模拟情况，从键盘输入选择指令来模拟用户程序的调用。

首先要建立一个“已打开文件表”，用来记录打开或建立文件的相关内容，结构如下图所示：

文件路径名	文件属性	起始盘块号	文件长度	操作类型	读指针		写指针	
					块号	块内地址	块号	块内地址

用数组模拟已打开文件表，数据结构定义如下：

```
#define n 5          //实验中系统允许打开文件的最大数量

typedef struct
{
    int dnum;          //磁盘盘块号
    int bnum;          //磁盘盘块内第几个字节
} pointer;             //已打开文件表中读、写指针的结构

typedef struct
{
    char name[20];     //文件绝对路径名
    char attribute;;    //文件的属性，用 1 个字节表示，所以此用 char 类型
    int number;         //文件起始盘块号
    int length;         //文件长度，文件占用的字节数
    int flag;           //操作类型，用“0”表示以读操作方式打开文件，用“1”表示以写操作方式打开文件

    pointer read;       //读文件的位置，文件打开时 dnum 为文件起始盘块号，bnum 为“0”
    pointer write;      //写文件的位置，文件刚建立时 dnum 为文件起始盘块号，bnum 为“0”，打开
```

文件时 dnum 和 bnum 为文件的末尾位置

```
}OFTLE;    //已打开文件表项类型定义  
  
struct  
{ OFILE file[n]; //已打开文件登记表  
    int length;    //已打开文件登记表中登记的文件数量  
}openfile;    //已打开文件登记表定义
```

无论上述哪种文件操作都会涉及到已打开文件表，对于已打开文件表主要是查找、删除和插入操作。下面分析所有常用的文件操作。

①建立文件（create_file）

用户要把一个新文件放到存储介质上前，首先调用文件系统的“建立”操作。

建立文件的主要工作就是检查文件目录，确认无重名文件后，寻找空闲登记项进行登记；寻找空闲存储块（至少一块）以备存储文件信息或存放索引表，最后应该填写已打开文件表。

实验中需要的参数比较少，只要有文件名和文件属性就可以，create_file（文件名，文件属性）。

实验中，建立文件时给出文件名和文件属性，文件属性如果是只读性质则不能建立；文件建立时根据给定的文件路径名进行查找，如果父目录不存在，建立文件失败；如果存在，查看有无重名文件，如果有，则提示该文件已存在，建立文件失败；如无重名文件，则为该文件建立文件目录，并分配给它一个磁盘块，最后填写目录和已打开文件表。

②打开文件（open_file）

用户要求使用一个已经存在的文件时，首先执行“打开文件”操作。可以在读或写一个文件时，默认打开该文件，将“打开”和“读或写”合二为一，不必非得先“打开”再“读或写”。

实验中，所需参数有文件名、操作类型（读或写），open_file（文件名，操作类型）。

实验中，打开文件首先要检查该文件是否存在，不存在，打开失败；如果文件存在，还要检查打开方式，确保不能以写方式打开只读文件；最后填写已打开文件表，若文件已经打开则不需要填写已打开文件表。

③读文件（read_file）

用户要求读文件信息时调用文件系统的“读文件”操作。

实验中，读文件的参数只需要文件名和读取长度，read_file（文件名，读取长度）。因为采用的是流式文件结构，所以读的长度用字节表示。

实验中，读文件操作的主要工作是查找已打开文件表中是否存在该文件；如果不存在，则打开后再读；然后检查是否是以读方式打开文件，如果是以写方式打开文件，则不允许读；最后从已打开文件表中读出读指针，从这个位置上读出所需要长度，若所需长度没有读完已经遇到文件结束符，就终止操作。实验中用“#”表示文件结束。

④写文件（write_file）

用户要求存取文件信息时调用文件系统的“写文件”操作。实验中，写文件的参数只需要文件名、存放准备写入磁盘信息的缓冲和写的长度，write_file（文件名，缓冲，写长度）。因为采用流式

文件结构，所以写长度用字节表示。

实验中，写文件操作的主要工作是查找已打开文件表中是否存在该文件，如果不存在，则打开后再写；如果存在，还要检查是否以写方式打开文件；如果不是写方式打开文件，不能写；最后从已打开文件表中读出写指针，从这个位置上写入缓冲中的数据。

写文件有两种情况，一种情况是建立文件后的写入，这种写比较简单，一边写一边申请空间即可完成；另一种情况是文件打开后的写入，这个比较复杂，存在着文件中间修改的问题。实验中，第二种情况只要求完成从文件末尾向后追加的功能。

⑤关闭文件（close_file）

用户对文件读写完毕后需要调用文件系统的“关闭文件”操作。

实验中，关闭文件的参数只需要文件名，close_file（文件名）。

实验中关闭文件，首先要看该文件是否打开，如果没有打开，就不用关闭；如果已经打开，则检查打开方式，如果是写方式打开的，要追加文件结束符，修改目录项；最后从已打开文件表中删除对应项。

⑥删除文件（delete_file）

用户认为文件没有必要保存时需要调用文件系统的“删除文件”操作。实验中，删除文件时参数只要文件名，delete_file（文件名）。

实验中，删除文件操作的主要工作是检查文件是否存在；不存在，操作失败；如存在，查找该文件是否打开，如果打开不能删除；如果没有打开，则删除文件目录项并归还文件所占磁盘空间。

⑦显示文件内容（typefile）

显示文件内容首先要找到该文件的目录登记项，如果文件不存在，指令执行失败；如果存在，查看文件是否打开，打开则不能显示文件内容；若没有打开，从目录中取出文件的起始盘块号，一块一块显示文件内容。

⑧改变文件属性（change）

改变文件属性，首先查找该文件，如果不存在，结束；如果存在，检查文件是否打开，打开不能改变属性；没有打开，根据要求改变目录项中属性值。

实验中，首先要系统初始化，包括建立文件模拟磁盘、初始化磁盘、初始化根目录为空目录项；然后，可以选择一项功能执行。

目录的操作命令：

①建立目录（md）

建立目录首先要找到建立目录的位置（父目录），然后查找该目录是否存在，如果父目录不存在，不能建立；如果存在，查找是否存在同名目录，存在，不能建立；不存在，则查找一个空目录项，为该目录申请一个盘块，并填写目录内容。

②显示目录内容（dir）

显示目录内容首先要找到该目录，如果目录不存在，指令执行失败；如果存在，一项一项显示目录内容。

③删除空目录（rd）

删除空目录首先要找到该目录，如果目录不存在，指令执行失败；如果存在，但是根目录或非空目录，显示不能删除，操作失败；若是非空子目录，则删除其目录项并回收对应空间。删除空目录的过程和删除文件的过程相似。

另外注意，对磁盘文件进行读操作时，需要磁盘的一个盘块读入主存后才能进行处理，对磁盘文件进行写操作时，要写满缓冲后才写入磁盘。所以模拟文件操作时，不能将整个模拟磁盘的内容同时读入主存，应该当需要模拟磁盘的某个盘块内容时，从对应文件中读出；修改后需要写回模拟磁盘。实验中就是用这种方法模拟磁盘的输入输出。

实验中需要定义两个数组 `buffer1` 和 `buffer2` 模拟缓冲。

实验中首先系统初始化，包括建立文件模拟磁盘，初始化磁盘和根目录初始化为空目录项，然后选择各个命令进行测试。

有能力的同学可在上述基础上，将磁盘文件系统改进为支持多级树型目录，支持相对路径，子目录可以任意长的文件系统。

题目六 模拟操作系统实现

一、课程设计目的

openEuler 是一个开源免费的 Linux 发行版系统，在 OpenEuler 操作系统中，使用高级编程语言（如 Java）来模拟操作系统的实现，加深对操作系统工作原理理解，进一步了解操作系统的实现方法，并可练习合作完成系统的团队精神和提高程序设计能力

二、小组人数

5 人一组共同完成模拟系统的实现。

三、编程语言

在 OpenEuler 操作系统中，使用高级编程语言（如 Java）来模拟并行发生的行为。图形界面或文字界面均可。

四、课程设计内容

模拟一个采用多道程序设计方法的单用户操作系统，该操作系统包括进程管理、存储管理、设备管理、文件管理和用户接口四部分。

五、课程设计具体内容和要求

1、文件管理和用户接口

文件管理和用户接口部分实现的主要是单用户的磁盘文件管理部分，包括文件的逻辑结构、物理结构、目录、磁盘分配回收、文件的保护和用户接口的实现。

（1）文件的逻辑结构

文件的逻辑结构采用流式结构；

文件用文本文件；

假设系统中只有两种文件，一种是存放任意字符的文件，一种是可执行文件。

可执行文件的内容就是模拟系统内进程的程序体，手工输入即可。

这里，“可执行文件”中的“命令”只有 5 种，包括：

x=? 给 x 赋值（一位数）

x++ x 加 1

x-- x 减 1

!?? !是“特命令（I/O）的前缀”，第一个?为 A,B,C 中的某个设备，第二个?为一位整数，表示使用设备的时间（由于没有实际设备，所以无法知道设备何时工作完成，所以假定一个数，这个数随着系统时间增加而递减（时间单位自定，例如：秒）。减到 0 时，认为是设备工作完成）；

end 表示“可执行文件”结束，同时将结果写入文件 out，其中包括

“可执行文件的路径名”和 x 的值。

每个可执行文件中可以包含多条同一类命令（指令）。

（2）磁盘模拟

用一个文件 disk 模拟磁盘，设磁盘的每个盘块 64 字节，模拟磁盘共有 128 块。第 0、1 块

存放文件分配表，第 2 块存放根目录，其余存放子目录和文件。（所以你创建的目录和流式文件不能太大太多，但至少包含 5 个目录和 15 个文件。注意：文件对磁盘块是独占的，即一个文件至少占据一个磁盘块，而一个磁盘块中不能同时存放两个文件的数据。）

（3） 目录结构

目录结构采用树型目录结构。

(a)目录项内容：

每个目录项 8 个字节，其中：

目录名或文件名：3 个字节；

扩展名：1 个字节（可执行文件扩展名为 e，目录没有扩展名）；

目录、文件属性：1 字节；

起始盘号：1 字节；

文件长度：2 字节（目录没有长度）。

(b)根目录

根目录位置固定，为磁盘第 2 块，大小固定，共可包含 8 个目录项，占用模拟磁盘第 2 块；

（c）子目录

位置不固定，大小不固定。

（4） 磁盘分配

磁盘的分配采用链接结构（显式链接）的分配方式。系统采用文件分配表方式记录磁盘空间的使用情况和链接结构的指针。（参考 MSDOS 的 FAT）

文件分配表中一项需要 1 字节，而磁盘有 128 块，因而 128 项，模拟磁盘空间中的第 0、1 块用来存放文件分配表。

（5） 用户接口

用户接口提供用户命令接口，接收用户从键盘键入的命令，要求实现以下命令：

（下面例子中的 \$ 只是命令提示符而已，和 UNIX 无关。除 aa 外，其余均为文件名。你可以根据自己实现方便或喜欢而自定义命令参数。）

需要实现的命令包括：

创建文件： create 例如 \$ create \aa\bb.e

删除文件： delete 例如 \$ delete \aa\yy

显示文件： type 例如 \$ type \zz

拷贝文件： copy 例如 \$ copy \xx \aa\yy

建立目录： mkdir 例如 \$ mkdir \dd

删除空目录： `rmdir` 目录非空时，要报错。

运行可执行文件：可执行文件的文件名。例如 `$bb.e` 可选实现的命令包括：

改变目录路径： `chdir`

删除目录： `deldir`（既可删除空目录又可删除非空目录）

移动文件： `move`

改变文件属性： `change`

磁盘格式化： `format`

磁盘分区命令： `fdisk`

上述命令在实际系统中都是需要建立进程才可以运行的，这里为简单起见，除运行可执行文件需要建立进程（在你的模拟系统里）外，其他命令执行不必在模拟系统中建立进程，可直接让 Windows 执行你编写的相应函数。

（6） 屏幕显示

如图 1，屏幕显示要求包括：

用户命令接口：用于系统运行时用户输入命令；

磁盘目录显示：要求显示磁盘的目录结构；（不需要画成“树状”）

磁盘使用情况：显示磁盘每一个磁盘块的空间是占用还是空闲。

2、存储管理

存储管理部分主要实现内存空间的分配和回收、存储保护。用数组模拟内存空间。

（1）模拟系统中，主存部分分为两部分，一部分是系统区，这里只存放进程控制块和内存分配表，一部分是用户区，这里主要是对用户区的管理。

系统区包括 PCB 区域、内存空间分配表；（长度自己定）

用户区用数组模拟，大小为 512 字节，存储管理采用动态分区存储管理方式。（最好能实现“可合并碎片”）

（2） 屏幕显示

如图 1，屏幕显示要求包括：内存使用情况示意图，以不同的颜色表示哪些区域已分配或未分配。

3、设备管理

设备管理主要包括设备的分配和回收。（可设一张“设备分配表”和设备等待队列）

（1）模拟系统中有 A、B、C 三种独占型设备，A 设备 1 个，B 设备 2 个，C 设备 2 个。（同一种设备的不同实体被认为是相同的）

（2）采用死锁预防方法来处理申请独占设备可能造成的死锁。

（3）屏幕显示如图 1 所示，屏幕显示要求包括：每个设备是否被使用，

哪个进程在使用该设备，哪些进程在等待使用该设备。

4、进程管理

进程管理主要包括进程调度，进程的创建和撤销、进程的阻塞和唤醒，中断作用的实现。

(1) 硬件工作的模拟

(a) 中央处理器的模拟

用函数 `CPU()`（该函数不能有参数）模拟中央处理器。

该函数主要负责解释“可执行文件”中的指令。（为简单计，用户接口部分实现的命令不必用 `CPU()`解释执行）

`x=?` 给 `x` 赋值一位数

`x++` `x` 加 1

`x--` `x` 减 1

`!??` 第一个?为 A,B,C 中的某个设备，第二个?为一位整数，表示使用设备的时间

`end` 表示文件结束。

`CPU` 只能解释指令寄存器 `IR` 中的指令。一个进程运行时要根据进程执行的位置，将对应指令存放到模拟的“指令寄存器”中。

(b) 主要寄存器的模拟（可选做）

用全局变量或数组模拟重要寄存器，如 `CPU` 重要寄存器，程序状态寄存器 `PSW`，指令寄存器 `IR`，程序计数器 `PC`，数据缓冲寄存器 `DR` 等。

(c) 中断的模拟（可选做）

I、中断的发现应该是硬件的工作，但在这里，用在函数 `CPU` 中检测 `PSW` 的方式来模拟。

在 `CPU()`函数中，每执行一条指令之前，先检查 `PSW`，判断有无中断，若有则先进行中断处理，然后再运行解释指令。`CPU` 函数应该不断循环执行。

II、模拟中断的种类和中断处理方式

① 程序结束（执行指令 `end` 形成的中断，软中断）：将结果写入文件 `out`，其中包括文件路径名和 `x` 的值，调用“进程撤销原语”撤销进程，然后进行进程调度；

② 时间片到（当相应寄存器值减到 0 时）：将正在运行进程的 `CPU` 现场（寄存器值即可，暂不考虑“栈”）保存在进程控制块中，然后进行进程调度；

③ I/O 中断（设备完成输入输出）：将输入输出完成的进程唤醒，同时将等待该设备的另一个进程唤醒。

(d) 时钟的模拟系统中的绝对时钟和相对时钟用全局变量模拟。系统时钟用来记录开机以后的时间，相对时钟用来存放进程可执行的时间片，在进程调度时设置初值，随系统时间的增值 1 而减值 1，减到 0 时，发出时钟中断。

这里的系统时钟并不是计算机的真正硬件时钟，这里所说的时间只是一个单位，例如秒或 500ms，可以使用程序语言里面的一些有关时间的函数来实现。

(2) 进程控制块

进程控制块内容包括进程标识符、主要寄存器组、进程状态、阻塞原因等。本模拟系统最多容纳 10 个进程块。PCB 区域用数组模拟。进程控制块根据内容的不同组成三个队列：空白 PCB 队列，就绪队

列和阻塞队列。正在运行的进程只有一个，系统初始时只有空白 PCB 队列。

(3) 进程调度

(a) 采用时间片轮转调度算法，时间片为 5。

(b) 进程调度函数的主要工作是：

将正在运行进程的现场（寄存器组）保存在该进程的 PCB 中；从就绪队列中选择一个进程；将这个进程的 PCB 中记录的各寄存器内容恢复到 CPU 各个寄存器内。（恢复现场）

(c) 闲逛进程

建立一个闲逛进程，当就绪队列为空时，系统调用该进程运行。当有进程就绪时，就调用就绪进程运行。闲逛进程什么有用的事也不做，只是起到系统能正常运转的作用。例如 Windows 中的 idle 进程不断将空闲内存清零。

(4) 进程控制

建立 4 个函数模拟进程创建、撤销、阻塞和唤醒四个原语

(a) 进程创建 create()，主要工作是：

第一步，申请空白进程控制块；

第二步，申请主存空间，申请成功，装入主存；

第三步，初始化进程控制块；

第四步，在屏幕上显示进程执行结果，进程撤销。

(b) 进程撤销 destroy()，主要工作是：

第一步，回收进程所占内存资源；

第二步，回收进程控制块；

第三步，在屏幕上显示进程执行结果，进程撤销。

(c) 进程阻塞 block()，主要工作是：

第一步，保存运行进程的 CPU 现场；

第二步，修改进程状态；

第三步，将进程链入对应的阻塞队列，然后转向进程调度。

(d) 进程唤醒 awake()

进程唤醒的主要工作是将进程由阻塞队列中摘下，修改进程状态为就绪，然后链入就绪队列。

(5) 屏幕显示

如图 1，屏幕显示要求包括：显示系统时钟；

显示正在运行进程的进程 ID、运行的指令、中间结果、相对时钟、寄存器内容；

显示就绪队列中进程 ID；显示阻塞队列中进程 ID。

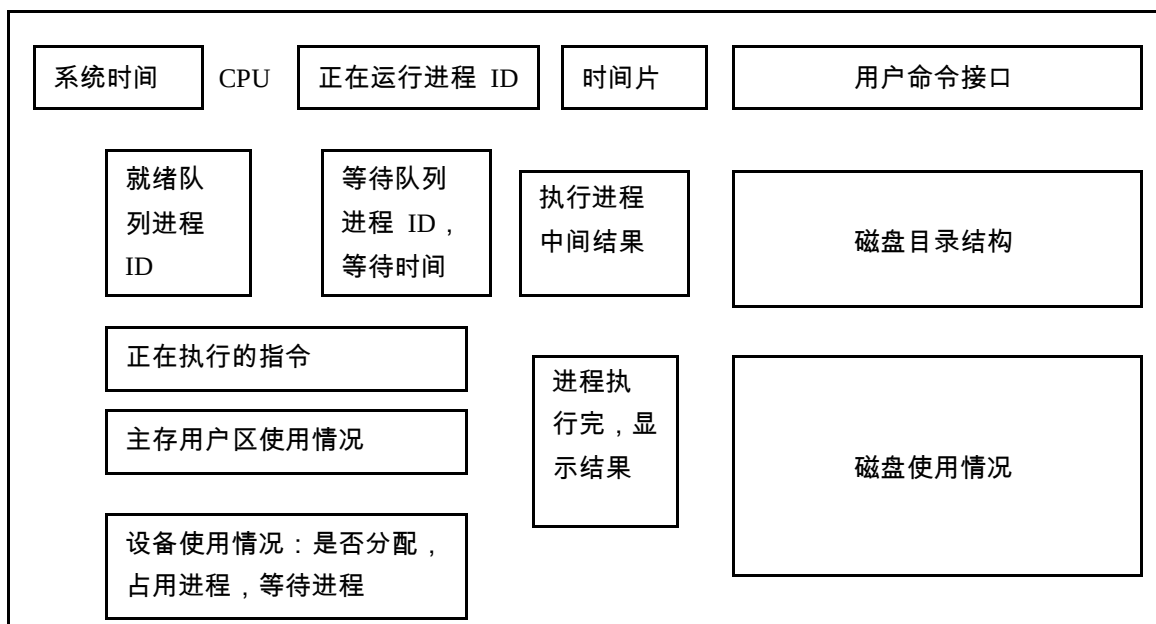


图 1 模拟操作系统的屏幕显示布局和内容

